SPARQL

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

SPARQL

https://github.com/mikel-egana-aranguren/ABD

CC BY

SPARQL

SPARQL **P**rotocol and **R**DF **Q**uery **L**anguage

SPARQL Endpoints públicos

Wikidata Query Service

DBpedia SPARQL

UniProt SPARQL

Open Data Euskadi

datos.gob.es

Recursos para aprender SPARQL

Learning SPARQL (libro)

Wikidata SPARQL Tutorial

Wikidata Query Service (con ejemplos)

SPARQL W3C (=HTML, !=SQL)

SPARQL 1.1 Query Language: consultas sobre grafos RDF en SPARQL

SPARQL 1.1 Update: modificar RDF mediante SPARQL

SPARQL 1.1 Protocol: ejecutar consultas SPARQL sobre HTTP

SPARQL 1.1 Graph Store HTTP Protocol: modificar grafos RDF mediante HTTP

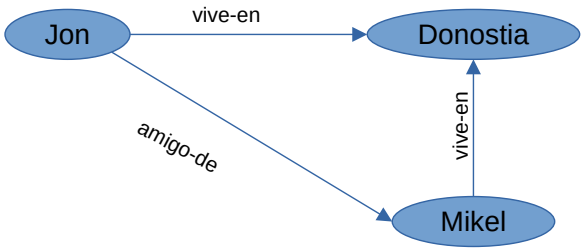
SPARQL 1.1 Service Description: describir endpoints SPARQL en RDF

SPARQL 1.1 Federated Query: combinar endpoints en una consulta SPARQL

GraphDB SPARQL 1.1 compliance (~~SPARQL 1.1 Service Description~~)

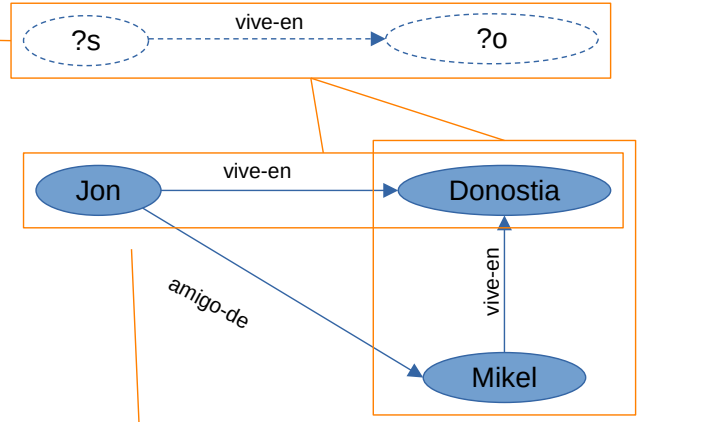
SPARQL

```
SELECt ?s ?o
WHERE {
      ?s vive-en ?o
}
```

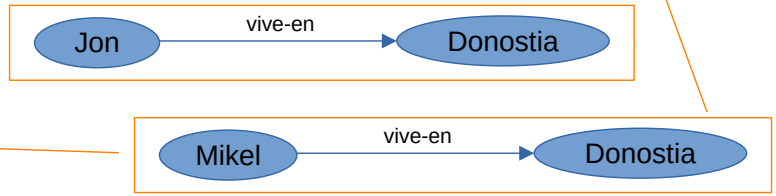


SPARQL

```
SELECt ?s ?o
WHERE {
      ?s vive-en ?o
}
```



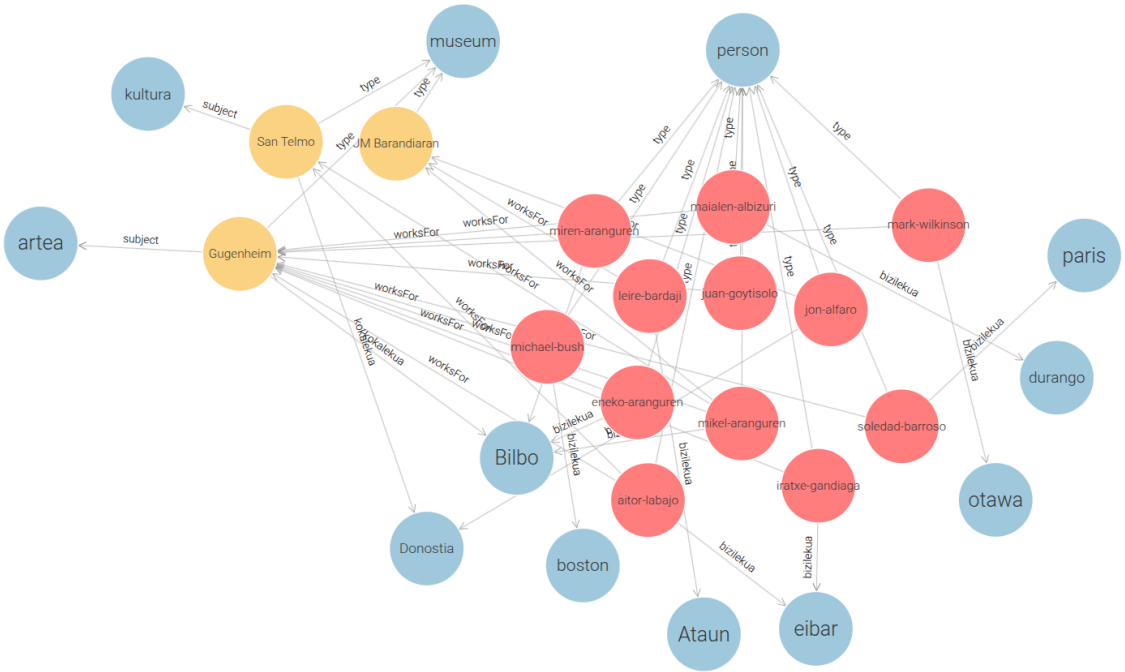
?s	?o
Jon	Donostia
Mikel	Donostia



Dataset museos

[ejemplos/Museoak.rdf]

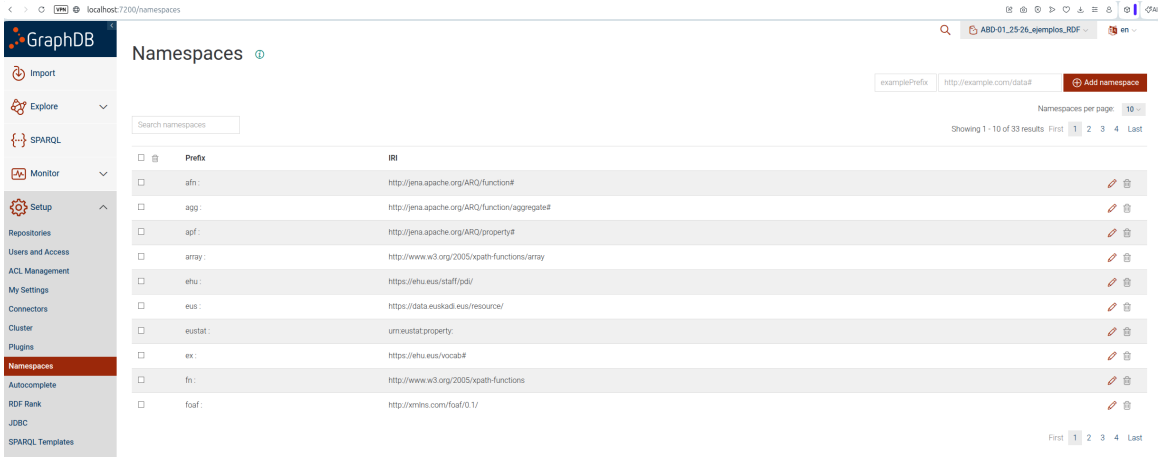
[ejemplos/./graphdb]



Dataset museos

```
# Prefixes
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX gip_prop: <http://gipuzkoa.eus/prop/>
PREFIX gip: <http://gipuzkoa.eus/resource/>
PREFIX dbpedia: <http://dbpedia.org/ontology/>
PREFIX gov: <http://vocab.data.gov/def/drm#>
```

GraphDB namespaces



Estructura de la consulta

```
# Variables que queremos recibir (Todas = "*")
SELECT ?sujeto ?clase

# Patrón del grafo que queremos extraer del grafo mayor
WHERE {
  ?sujeto gip_prop:bizilekua gip:donostia .
  ?sujeto rdf:type ?clase
}
```

OPTIONAL

```
SELECT ?langile ?bizilekua

WHERE {
  # Tiene que ser una persona
  ?langile rdf:type foaf:person .

  # Tiene que vivir en algun sitio
  ?langile gip_prop:bizilekua ?bizilekua
}
```

OPTIONAL

```
SELECT ?langile ?bizilekua

WHERE {
  # Tiene que ser una persona
  ?langile rdf:type foaf:person .

  # Puede vivir en un sitio o no
  OPTIONAL {
    ?langile gip_prop:bizilekua ?bizilekua
  }
}
```

UNION

```
SELECT ?entitatea ?izena

WHERE {
  { ?entitatea rdfs:label ?izena }
  UNION
  { ?entitatea foaf:name ?izena }
}
```

Ordenar resultados

```
SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}

ORDER BY DESC (?izena) # Puede ser DESC o ASC
```

Limitar resultados

```
SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}

ORDER BY DESC (?izena)
LIMIT 3
```

Paginación

```
SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}

ORDER BY DESC (?izena)
LIMIT 3
OFFSET 3 # OFFSET 6
```

Filtrar resultados

```
SELECT ?museoa ?langile_kop

WHERE {
  ?museoa gip_prop:kopurua ?langile_kop .
  FILTER (?langile_kop > "800"^^xsd:int)
}
```

Filtrar resultados

```
SELECT ?langile ?izena

WHERE {
  ?langile foaf:name ?izena .
  FILTER regex(?izena,'Mi.*')
}
```

Filtrar resultados

Lógica: !, &&, ||

Cálculos: +, -, *, /

Comparaciones: =, !=, >,<

Filtrar resultados

Tests SPARQL: isURI, isBlank, isLiteral, bound

Acceder a datos: str, lang, datatype

Más: sameTerm, langMatches, regex, …

Evitar duplicados

```
SELECT ?lantokia # DISTINCT
WHERE {
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?lantokia
}
```

ASK

```
ASK
WHERE {
  ?person gov:worksFor gip:gugenheim
}
```

DESCRIBE

```
DESCRIBE <http://gipuzkoa.eus/resource/mikel-aranguren>
```

[Visual]

GRAFOS

Grafo: conjunto de triples

El conjunto entero se identifica con una URI (diferente de la de los datos)

Todas las Triple Stores tienen un Default Graph

GRAFOS

```
INSERT DATA{
  GRAPH <http://example/bookStore> {
    <http://example/book1> gip_prop:price 42
  }
}
```

```
SELECT *
FROM <http://example/bookStore> # Probar sin URI grafo
WHERE {
  ?s ?p ?o .
}
```

Funciones de agregación

COUNT: contar elementos

SUM: suma de valores

AVG: promedio

Funciones de agregación

MIN / MAX: mínimo y máximo

GROUP_CONCAT: concatenar valores

SAMPLE: obtener un valor de ejemplo

COUNT

```
SELECT (COUNT(?person) AS ?total)
WHERE {
  ?person rdf:type foaf:person
}
```

GROUP BY

Agrupar resultados por valores de una variable

```
SELECT ?ciudad (COUNT(?museo) AS ?numMuseos)
WHERE {
  GRAPH <urn:count> {
    ?museo rdf:type dbpedia:museum ;
    gip_prop:kokalekua ?ciudad .
  }
}
GROUP BY ?ciudad
ORDER BY DESC(?numMuseos)
```

[ejemplos/Museoak-count.ttl]

HAVING

Filtrar grupos después de la agregación

```
SELECT ?ciudad (COUNT(?museo) AS ?numMuseos)
WHERE {
  GRAPH <urn:count> {
    ?museo rdf:type dbpedia:museum ;
    gip_prop:kokalekua ?ciudad .
  }
}
GROUP BY ?ciudad
HAVING (COUNT(?museo) > 1)
ORDER BY DESC(?numMuseos)
```

BIND

Asignar valores a variables dentro de la consulta

```
SELECT ?person ?fullName
WHERE {
  GRAPH <urn:name-surname>{
    ?person foaf:firstName ?first ;
    foaf:lastName ?last .
  }
BIND(CONCAT(?first, " ", ?last) AS ?fullName)
}
```

[ejemplos/Museoak-name-surname.ttl]

Funciones de cadenas

CONCAT: concatenar strings

STRLEN: longitud de string

SUBSTR: subcadena

UCASE / LCASE: mayúsculas / minúsculas

Funciones de cadenas

STRSTARTS / STRENDS / CONTAINS: búsqueda en strings

REPLACE: reemplazar texto

REGEX: expresiones regulares

REGEX

```
SELECT ?museo ?nombre
WHERE {
  ?museo rdf:type dbpedia:museum ;
    rdfs:label ?nombre .
  FILTER REGEX(?nombre, "Gugen")
}
```

Funciones numéricas

ABS: valor absoluto

ROUND / CEIL / FLOOR: redondeo

RAND: número aleatorio

Operadores: +, -, *, /

Funciones de fecha/hora

NOW(): fecha y hora actual

YEAR / MONTH / DAY: extraer componentes

HOURS / MINUTES / SECONDS: hora específica

VALUES

Proporcionar valores inline para variables (Útil para consultas parametrizadas)

```
SELECT ?museo ?obras
WHERE {
  VALUES ?museo {
    <http://gipuzkoa.eus/resource/gugenheim>
    <http://gipuzkoa.eus/resource/san-telmo>
  }
  ?museo gip_prop:kopurua ?obras
}
```

MINUS

Excluir resultados que coinciden con un patrón: todos los museos menos los museos que tienen más de 300 obras

```
SELECT ?museo ?obras
WHERE {
  ?museo rdf:type dbpedia:museum .
  MINUS {
    ?museo gip_prop:kopurua ?obras .
    FILTER (?obras > 300)
  }
}
```

NOT EXISTS

Filtrar basándose en la no existencia de un patrón

EXISTS vs MINUS

EXISTS/NOT EXISTS: filtro, no elimina bindings

MINUS: elimina soluciones completas

NOT EXISTS es más eficiente para validación

MINUS es más útil para exclusión de patrones complejos

Property Paths

^: inverso (ex:parent^ encuentra hijos)

: cero o más (ex:knows para red de conocidos)

+: uno o más (ex:subClassOf+ para jerarquía)

!: cualquier relacion menos esa

Property Paths

?: opcional (cero o uno)

|: alternativa (ex:email|ex:phone)

/: secuencia (ex:livesIn/ex:country)

Property paths

```
SELECT ?person ?city
WHERE {
  ?person rdf:type foaf:person .
  ?person gov:worksFor/gip_prop:kokalekua ?city .
}
```

SPARQL 1.1 Update

Estándar del W3C (=HTML, !=SQL)

DELETE DATA

```
DELETE DATA {
  <http://gipuzkoa.eus/resource/aitor-labajo>
  rdf:type foaf:person
}
# DESCRIBE <http://gipuzkoa.eus/resource/aitor-labajo>
```

DELETE WHERE

```
DELETE {?person rdf:type foaf:person}
WHERE {?person foaf:name ?name}

#SELECT * WHERE {
#  ?person rdf:type foaf:person
#}
```

INSERT DATA

```
INSERT DATA {
  gip:aitor-labajo rdf:type gip:hiritar
}
# DESCRIBE gip:aitor-labajo
```

INSERT WHERE

```
INSERT {
  gip:jon-alfaro rdf:type ?type .
}
WHERE {
  gip:aitor-labajo rdf:type ?type .
}
# DESCRIBE <http://gipuzkoa.eus/resource/jon-alfaro>
```

DELETE/INSERT combinado (UPSERT)

Actualizar email de manera transaccional:

```
DELETE { ?person ex:email ?oldEmail }
INSERT { ?person ex:email ?newEmail }
WHERE {
  ?person foaf:name "Mikel" ;
    ex:email ?oldEmail .
  BIND("mikel@new.com" AS ?newEmail)
}
```

CONSTRUCT

```
CONSTRUCT {
  ?langile rdf:type gip:gugenheim_langilea
}
WHERE {
  ?langile gov:worksFor gip:gugenheim
}
```

LOAD

(=HTML, !=SQL)

```
LOAD <https://rest.uniprot.org/uniprotkb/P01308.rdf>
INTO GRAPH <urn:uniprot>
```

CLEAR

```
CLEAR GRAPH <urn:uniprot>

# CLEAR DEFAULT

# CLEAR ALL
```

DROP

```
DROP GRAPH <http://example.org/myGraph>

# DROP ALL
```

COPY / MOVE / ADD

COPY: copiar un grafo a otro (sobreescribe destino)

MOVE: mover grafo (copia + elimina origen)

ADD: añadir triples al destino (sin sobreescribir)

COPY / MOVE / ADD

```
COPY <urn:name-surname>
TO <urn:nombre-apellidos>

COPY DEFAULT
TO <urn:backup>
```

SPARQL 1.1 Protocol

Estándar del W3C (=HTML, !=SQL)

SPARQL 1.1 Protocol

HTTP GET con parámetro query: `GET /sparql?query=SELECT...`

HTTP POST (para queries largas): `POST /sparql` con body URL-encoded

Content negotiation (Accept header): application/sparql-results+json, text/csv, application/rdf+xml, text/turtle

[ejemplos/post.sh]

SPARQL 1.1 Protocol

Python: RDFLib, SPARQLWrapper (ejemplos/sparql.py)

Java: Apache Jena, RDF4J

SPARQL 1.1 Service Description

Estándar del W3C (=HTML, !=SQL)

SPARQL 1.1 Service Description

Proporciona metadatos sobre endpoints SPARQL para ayudar a los clientes:

- **Descubrir capacidades**: ¿Qué funcionalidades soporta? (agregaciones, property paths, negación, etc.)
- **Encontrar datasets disponibles**: ¿Qué Named Graphs están disponibles?
- **Determinar formatos soportados**: JSON, XML, CSV, …
- **Ver regímenes de inferencia**: RDFS, OWL, …

SPARQL 1.1 Service Description

```
curl -H "Accept: text/turtle"
https://dbpedia.org/sparql/
```

[ejemplos/dbpedia_service_description.ttl]

SPARQL 1.1 Federated Query

Estándar del W3C (=HTML, !=SQL)

Consultas Federadas

Permiten consultar múltiples endpoints SPARQL en una sola query

Combinar datos de fuentes distribuidas usando la palabra clave `SERVICE`

Consultas federadas

```
INSERT DATA {
  <http://gipuzkoa.eus/resource/bilbo> owl:sameAs <http://dbpedia.org/resource/Bilbao> .
}
```

```
SELECT ?pop
WHERE {
  gip:bilbo owl:sameAs ?town .
  SERVICE <https://dbpedia.org/sparql/> {
    ?town <http://dbpedia.org/ontology/populationTotal> ?pop .
  }
}
```

SPARQL 1.1 Graph Store HTTP Protocol

Estándar del W3C (=HTML, !=SQL)

Operaciones

Gestionar grafos RDF mediante HTTP:

- **GET**: Recuperar un grafo específico
- **PUT**: Reemplazar un grafo completo (Destructivo)
- **DELETE**: Eliminar un grafo
- **POST**: Añadir triples a un grafo

Operaciones: Parámetros

Endpoint base: según configuración

Parámetros:

- `graph=<URI>` : Especifica el grafo con nombre
- Sin `graph` : Operación sobre el grafo por defecto

Content-Type: RDF/XML, Turtle, N-Triples, etc.

[ejemplos/graph_store_protocol.sh]

Ventajas

- **RESTful**: Interfaz simple basada en HTTP estándar
- **Automatización**: Integración con scripts y herramientas CLI
- **Actualizaciones sin SPARQL UPDATE**: Alternativa a INSERT/DELETE
- **Gestión de versiones**: Reemplazar versiones completas de grafos

Graph Store vs SPARQL UPDATE

Aspecto	Graph Store HTTP	SPARQL UPDATE
Granularidad	Grafo completo	Triples individuales
Interfaz	REST/HTTP	SPARQL query
Uso típico	Reemplazar versiones enteras	Modificaciones precisas
Formato	RDF (TTL, RDF/XML, etc.)	SPARQL INSERT/DELETE

GraphDB Ontop: Virtualización de datos relacionales

Ontop es un plugin de GraphDB que permite consultar bases de datos relacionales (SQL) mediante SPARQL

Los datos **no se copian** al triplestore: se acceden en tiempo real (virtualización)

Ontop traduce las consultas SPARQL a SQL automáticamente: El usuario consulta con SPARQL sin saber que los datos están en SQL

GraphDB Ontop: Arquitectura

- **Base de datos relacional**: la fuente de datos SQL existente
- **Ontología**: define el vocabulario RDF (clases y propiedades)
- **Mapping R2RML**: mapeo entre tablas/columnas SQL y el grafo RDF
- **Ontop/GraphDB**: motor que traduce SPARQL → SQL en tiempo real

R2RML: RDB to RDF Mapping Language

R2RML es un estándar del W3C para definir mapeos de bases de datos relacionales a RDF

- **Triples Map**: define cómo generar triples RDF a partir de una tabla o consulta SQL
- **Logical Table**: la tabla SQL o consulta SQL de origen
- **Subject Map**: cómo generar la URI del sujeto
- **Predicate-Object Map**: cómo generar predicados y objetos

R2RML: Ejemplo de mapeo

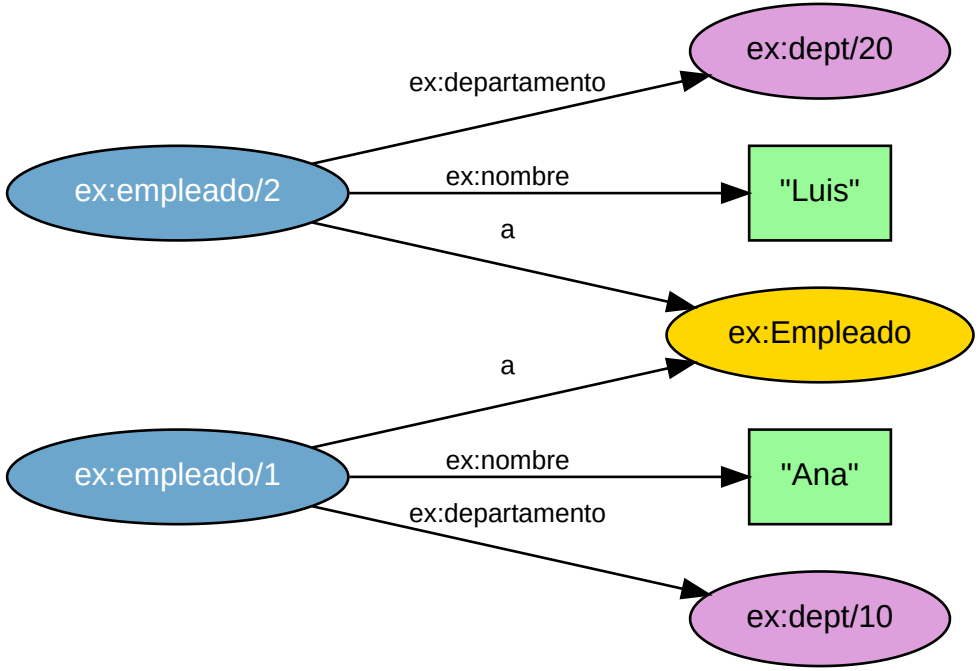
Tabla SQL: `empleados(id, nombre, dept_id)`

```
<#TriplesMap1> a rr:TriplesMap ;
  rr:logicalTable [ rr:tableName "empleados" ] ;
  rr:subjectMap [
    rr:template "http://ejemplo.org/empleado/{id}" ;
    rr:class ex:Empleado
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:nombre ;
    rr:objectMap [ rr:column "nombre" ]
  ] ;
    rr:predicateObjectMap [
      rr:predicate ex:departamento ;
      rr:objectMap [
        rr:template "http://ejemplo.org/dept/{dept_id}" ]
  ] .
```

R2RML: Resultado del mapeo

id	nombre	dept_id
1	Ana	10
2	Luis	20

R2RML: Resultado del mapeo



GraphDB Ontop: Ventajas

- No se duplican datos: los datos siguen en la BD relacional
- Datos siempre actualizados (acceso en tiempo real)
- Estándar W3C (R2RML): portable entre motores

GraphDB Ontop: Limitaciones

- Rendimiento depende de la BD relacional subyacente
- No todas las funciones SPARQL tienen traducción directa a SQL

GraphDB Explain Plan

GraphDB join flow: evaluar joins entre patrones con variables compartidas

(SPARQL: inner join implicito)

GraphDB Explain Plan: analizar el orden de evaluación optimizado

Se activa añadiendo `FROM onto:explain` a la consulta

(`onto:measure` provee los tiempos de ejecución)

GraphDB Explain Plan

```
PREFIX onto: <http://www.ontotext.com/>
SELECT * FROM onto:explain # onto:measure
WHERE {
  ?s ?p ?o.
}
```

```
SELECT ?s ?p ?o
{

  { # ----- Begin optimization group 1 -----

    ?s ?p ?o . #    Collection size: 157    Predicate collection size: 157  Unique subjects: 144    Unique objects: 87  Current complexity: 157

  } # ----- End optimization group 1 -----
  # ESTIMATED NUMBER OF ITERATIONS: 157

}

# NOTE: Optimization groups are evaluated one after another exactly in the given order.
# If there are too many optimization groups consisting of a single statement pattern,
# then one should try to relocate the following clauses by hand:
# VALUES, BIND, OPTIONAL, property paths with '*' and/or '+' (the latter can be also surrounded with brackets).
# Sub-SELECTs will always be evaluated first.
```

GraphDB Explain Plan

Optimization group: grupo de triples, ordenados segun optimización

Collection size: triples que coinciden con el patrón

Predicate collection size: total de triples en el repositorio que tienen este predicado

Unique subjects: número de sujetos únicos que coinciden con el patrón

GraphDB Explain Plan

Unique objects: número de objetos únicos que coinciden con el patrón

Current complexity: la complejidad (el número de búsquedas atómicas en el índice) que la base de datos necesitará realizar para ese patrón

Estimated number of iterations: las iteraciones que se ejecutarán para este grupo

GraphDB Explain Plan

Orden de triples dentro de grupo por selectividad

Selectividad de un patrón: collection size (Overload de predicado), joins (sujetos/objetos unicos), variables resueltas en patrón anterior

Explain Plan

```
PREFIX onto: <http://www.ontotext.com/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX gov: <http://vocab.data.gov/def/drm#>

SELECT * FROM onto:explain
WHERE {
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?museo .
  ?person foaf:name ?nombre .
}
```

Explain Plan: Orden de patrones

foaf:name: collection size 13 (el más bajo) pero además predicate collection size también 13: foaf:name solo se usa en esos triples, muy selectivo

gov:worksFor collection size 16, pero ?person ya esta resuelto

rdf:type collection size 54, con 45 sujeto unicos y objetos unicos

Complejidad = collection size / selectividad (Incluyendo variables resueltas)

GraphDB Explain Plan

Triples dentro de grupo: se optimizan automáticamente

Orden de grupos: no se optimiza automáticamente, se evalúan en el orden en que aparecen en la consulta (el usuario debe optimizarlos)

Los grupos aparecen cuando se rompe el flujo de joins

Rompen el flujo de joins

OPTIONAL: hay que resolver el left join completo primero y luego la parte opcional

UNION: se evalúan independientemente y luego se combinan

sub-SELECT: siempre se evalúa primero

FILTER, BIND, VALUES: dependen de variables anteriores

Rompen el flujo de joins

Property paths con * o +: expansión recursiva

Patrones desconectados sin variables compartidas

Explain Plan: FILTER temprano vs tardío

Consulta A: FILTER al final (ineficiente)

```
SELECT * FROM onto:explain
WHERE {
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?museo .
  ?person foaf:name ?nombre .
  FILTER regex(?nombre, "Mi.*")
}
```

Primero genera TODAS las combinaciones, después filtra

Explain Plan: FILTER temprano vs tardío

Consulta B: FILTER justo después de ?nombre (eficiente)

```
SELECT * FROM onto:explain
WHERE {
  ?person foaf:name ?nombre .
  FILTER regex(?nombre, "Mi.*")
  ?person rdf:type foaf:person .
  ?person gov:worksFor ?museo .
}
```

Filtra pronto → menos resultados intermedios → menos joins

Explain Plan: Impacto de OPTIONAL

```
SELECT * FROM onto:explain
WHERE {
  ?museo rdf:type dbpedia:museum .
  ?museo rdfs:label ?nombre .
  OPTIONAL {
    ?museo gip_prop:kopurua ?obras .
  }
  OPTIONAL {
    ?museo gip_prop:kokalekua ?ciudad .
  }
}
```

Cada OPTIONAL añade un Left Join → más costoso

Explain Plan: Evitar OPTIONAL innecesario

Si sabemos que todos los museos tienen kokalekua y kopurua:

```
SELECT * FROM onto:explain
WHERE {
  ?museo rdf:type dbpedia:museum .
  ?museo rdfs:label ?nombre .
  ?museo gip_prop:kopurua ?obras .
  ?museo gip_prop:kokalekua ?ciudad .
}
```

Sin OPTIONAL: Inner Join, más eficiente